```
t.toString() + "\nImplicit toString() call: " + t;
System.exit( 0 );
}
}
```

In this next line, you see how we are calling two different methods of our new class. We're calling toUniversalString() and toString(). One curious thing is this naked reference " t " sitting out here by itself. What does that do? Well, anytime you concatenate an object to a String, you automatically call its toString() method.

4.9 Varieties of Methods: details

In applications we have created so far, there has only been the method main. In the applets we have created, there have been several standard methods: `init()`, `start()` and `paint()`. Applets have other standard methods, as we know. The class we created `Time1` is neither an application nor an applet. Time1 cannot execute unless another program first instantiates it. Encapsulating data and the methods used to access that data is the central role of the class in object oriented programming. With this class, we make the member variables private. We create methods used to access the member variables. There are several different varieties of methods, and each kind has a different sort of job to do.

Constructors—public methods used to initialize a class.

Accessors—public methods (gets) used to read data.

Mutators—public methods (sets) used to change data.

Utility—private methods used to serve the needs of other public methods.

Finalizers—protected methods used to do termination housekeeping.

The Constructor is named exactly the same as the class. It is called when the new keyword is used. It cannot have any return type, not even void. It instantiates the object. It initializes instance variables to acceptable values. The "default" Constructor accepts no arguments. The Constructor method is usually overloaded. The overloaded Constructor usually takes arguments. That allows the class to be instantiated in a variety of ways. If the designer neglects to include a constructor, then the compiler creates a default constructor that takes no arguments. A default constructor will call the Constructor for the class this one extends.

```
public Time2()
{
setTime( 0, 0, 0 );
```